

COURSE PROJECT

GenAI System Design Project

Guidelines, templates and checklist for designing one AI system that solves a real business problem.

SUBMISSION DEADLINE

Sun · 15 Nov 2026

PROJECT PRESENTATIONS

Tue · 17 Nov 2026

The model is the engine. The harness decides what it can see, do and be trusted with.

IN THIS PACK

Contents

Start with the guidelines, write the project deck, detail the use cases, condense it into the one-pager, then verify everything against the checklist.

01 GUIDELINES — 3

Guidelines

Purpose, thinking path, building blocks, deliverables and the rationale behind them.

02 TEMPLATE — 6

Project deck (PRD)

The PRD: problem, solution, big picture, use cases, risks, conclusions, component catalog and the pitch.

03 TEMPLATE — 10

Use case guidelines

Ten topics that turn each use case from a story into a testable, observable design.

04 TEMPLATE — 13

One-pager

The two-minute read for a sponsor: problem, solution, risks and evals.

05 CHECKLIST — 15

Checklist

What to verify before you hand in, and the questions to challenge any design.

! **Disclaimer:** these are guidelines, not strict rules. They give a rationale and a structure for learning; adapt them to your problem and explain why.

GUIDELINES

01 Guidelines

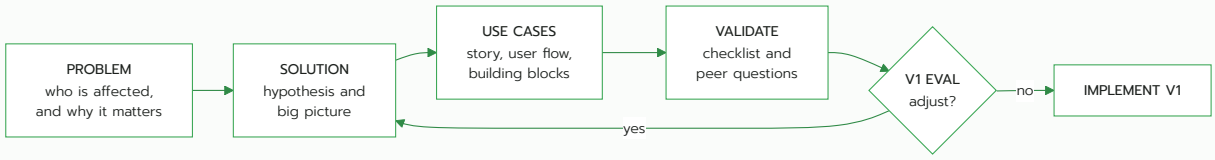
Purpose

Design *one AI system that solves a real business problem*, with GenAI as the main focus. Task-specific models (classification, extraction, forecasting) are welcome, alone or combined with GenAI. You do not have to build it; you have to show the business case and the reasoning that makes it trustworthy.

> Which business problem is worth solving with AI, what must exist around the model for it to work reliably, and how will we know it creates value?

The model is the engine. The **harness** (the building blocks around it) decides what the model can see, do and be trusted with.

Thinking path



Building blocks (vocabulary)

	BLOCK	WHAT IT DOES	EXAMPLES
M	Models	The AI engines	GenAI model (LLM); task-specific model (classifier, extractor, forecaster)
C	Context	What the model needs to know	Documents, RAG index, repositories, knowledge graph, schemas, memory, prompts & skills, policies
T	Tools	What the model can fetch, compute or do	Tool, API, MCP server (a shared connector); read or act
W	Workflow	Flow where **your code** decides the path	Fixed steps, routing by type, draft then check
A	Agent	Flow where **the model** decides the path	Think → act → observe loop; single or multi-agent

	BLOCK	WHAT IT DOES	EXAMPLES
G	Guardrails	Checks in code around the model	Input, execution, output
E	Evals	Tests that prove quality	Code checks, an LLM as judge, human review; accuracy for classifiers
X	Experiments	Compare design choices	A/B on a fixed dataset, small live pilot
O	Observability	See what it did, why, how fast, at what cost	Traces, errors, quality signals

Flow patterns, simplest first: prompt → model, workflow, agent. **Success metrics** measure business value (time, cost, revenue, risk); **evals** measure the system's quality.

Deliverables

DELIVERABLE	WHAT IT IS	SIZE	GUIDE
Project deck (PRD)	The full design: what, for whom, why, and how each use case works	5–10 pages	`template-project-deck.md`, `template-use-cases.md`
Pitch	The project deck's story for a decision maker	5 slides	`template-project-deck.md` (Pitch section)
Use Cases	List and detail some use cases	4–10 pages	`template-use-cases.md`
One-pager	The high-level solution a sponsor reads in two minutes	1 page	`template-one-pager.md`
Prototype	A toy that proves one or two key concepts on the riskiest use case	—	—

Write the project deck first; the pitch and the one-pager select from it. Verify everything with the checklist (`checklist.md`).

Submission and presentation

WHAT	WHEN
Submission deadline	**Sunday, 15 November 2026**
Project presentations	**Tuesday, 17 November 2026**

How to submit

- **Send to:** fperes@novaims.unl.pt and roberto@novaims.unl.pt

- **Subject:** `[AI-intro-project] [group name]`
- **Attachment:** one zip file with the project deck, the use cases and the one-pager.

> **Prototype:** do **not** send it. You will demonstrate it live at the presentations on 17 November. You can include screenshots of it in the project deck.

Rationale

1. **One bounded business problem:** one process, one user group, one measurable business outcome.
2. **Simplest option that works:** rules or a task-specific model when they suffice; GenAI where language and judgement are needed; workflow before agent.
3. **Hardest use case first:** design and prototype the riskiest one, not the easiest demo.
4. **Only the blocks you need:** skip a block and say why in one line.
5. **Accountability stays with a person,** never with a model or a vendor.
6. **Guardrails are code, not requests:** the prompt guides; the checks enforce.
7. **No eval, no evidence:** every use case has a test set and a threshold.
8. **Running is not right:** observe quality, not only uptime.
9. **State your assumptions,** especially the one that breaks the design if wrong.

TEMPLATE

02 Project deck (PRD)

The project deck is the PRD (Product Requirements Document): what the system does, for whom, why, and how each use case works. 6–10 pages. Terms follow the building blocks vocabulary in the guidelines.

TOPICS 1 Problem | 2 Solution and target users | 3 Big picture | 4 Use cases | 5 Risks and guards | 6 Conclusions | 7 Component catalog

1 Problem

What problem matters, and how do we know?

> **[User]** struggles to **[task]** because **[cause]**, resulting in **[measurable consequence]**.

- **Baseline** in business terms (time or cost per case, error rate, volume, € at stake), **scope** (one process, one user group) and **non-goals**.
- **Which AI approach:** pick per task; a solution can combine several.

OPTION	FITS WHEN	USED FOR / REJECTED BECAUSE
Rules / process change	Stable, explicit, deterministic task	
Task-specific model	Classify, extract, score or forecast with clear labels or history	
GenAI: prompt → model or workflow	Unstructured content, language, synthesis	
GenAI: agent	Next step depends on what it finds	

2 Solution and target users

What do we believe will work, and for whom?

> We believe that using **[AI capability]** to help **[users]** perform **[task]** will improve **[outcome]** from **[baseline]** to **[target]**, while controlling **[main risks]** through **[key guards]**.

SUCCESS METRIC	BASELINE	TARGET	HOW MEASURED

- **Target users:** who they are, what they need, what hurts today.
- Mix **value** (time, cost, revenue, risk avoided), **quality** (correctness) and **adoption** (% AI suggestions accepted).
- Include the running cost (per case or per month) against the value created.

3 Big picture

How does the whole system fit together?

- One diagram: users → guardrails → flow (workflow / agent) with context and tools → human decision → action, with evals and observability around it. Start from the one-pager diagram.

4 Use cases

In the project deck, include only the use cases summary table.

- Use case list (3–5), with the riskiest marked ★:

#	USE CASE	TARGET USER	TRIGGER	EXPECTED OUTPUT	PRIORITY

Create an appendix with the full details of all use cases. This can be organized either as a single file containing all use cases or as a separate file for each use case.

For more information about the use cases, see the use case guidelines.

5 Risks Analysis

What can go wrong, and what stops it?

RISK / ASSUMPTION	IMPACT IF WRONG	GUARD OR VALIDATION	OWNER

- Typical risks: plausible but wrong answers, data leaks, regulation (GDPR, EU AI Act), over-reliance, stale knowledge, prompt injection, low adoption.
- Mark the **key assumption**: the one that breaks the design if wrong.

6 Conclusions

What is proven, and what comes next?

- **Now:** what is proven (evals, prototype) vs. still a hypothesis.
- **Next:** the decision you need (budget, pilot, team), the experiments to run, the V1 scope.

7 Component catalog

Which components does the system use?

Every component, listed once. Use cases and the one-pager refer to it by ID.

ID	NAME	TYPE	DESCRIPTION	USED BY
RAG-01	Policy search	RAG index	Vector index over policy PDFs; metadata: date, product	UC1, UC2
MOD-01	Claim classifier	Model (task-specific)	Classifies incoming claims by type; trained on 2 years of history	UC1
TL-01	Order lookup	Tool (read)	Reads order status from the ERP API	UC2
WF-01	Claim triage	Workflow	Classify → retrieve → draft → check	UC1

Types (ID prefix):

- **Context:** Documents `DOC` · RAG index `RAG` · Repository `REPO` · Knowledge graph `KG` · Schema `SCH` · Memory `MEM` · Prompt / skill `PRM` · Policy `POL`
- **Model** `MOD` (GenAI or task-specific) · **Tool** `TL` (read or act) · **Workflow** `WF` · **Agent** `AG` · **Guardrail** `GR` · **Eval** `EV` · **Experiment** `XP` · **Observability** `OB`

Pitch (5 slides)

#	SLIDE	FROM PROJECT DECK
1	Problem	Problem, baseline, which AI approach
2	Solution and use cases	Hypothesis, target users, use cases (★ riskiest)
3	Big picture	Diagram; zoom on the riskiest use case
4	Risks and guards	Top risks, key assumption, guards

#	SLIDE	FROM PROJECT DECK
5	Evals and next steps	Success metrics, eval thresholds, evidence, decision asked

One message per slide; the title states the conclusion. Show numbers.

TEMPLATE

03 Use case guidelines

Feeds topics 3 (use case list) and 4 (use cases) of the project deck. Each use case document follows the topics below, in order. Refer to components by their component catalog ID (project deck, topic 7) (e.g. `RAG-01`, `TL-02`).

Only the blocks you need: skip a topic with a one-line reason (e.g. "Tools: none, read-only answers from documents").

TOPICS 1 Story · 2 User flow · 3 Context · 4 Tools · 5 Flow and models · 6 Guardrails · 7 Evals · 8 Observability · 9 Experiments · 10 Risks and open questions

1 Story

Who needs what, and what changes for them?

> "As a **[user]**, when **[trigger]**, I want **[outcome]**, so that **[value]**."

2 User flow

What happens from trigger to outcome, and who does each step?

STEP	HUMAN	AI	TOOL / SYSTEM	AUTONOMY
1				Suggests / acts after approval / acts alone

- Mark **where a human decides** and **what happens when a step fails** (fallback, escalation, retry).
- More autonomy needs more guardrails and evals.

3 Context (C)

What must the model know at each step?

STEP	NEED	COMPONENT (ID)	FRESHNESS & ACCESS
		`RAG-01`	

- Include memory (what is kept, what is never stored) and the **output schema** (`SCH`) code can validate, or state "free text for a human".

- Relevance beats volume: too much context degrades answers.

4 Tools (T)

What must it fetch, compute or do that the model cannot?

TOOL (ID)	PURPOSE	READ / ACT

- The model only ***requests*** a call; your code runs it and validates the arguments.
- Act tools (refund, email) need limits or approval.

5 Flow and models (W / A / M)

Who decides the next step, and which model does each AI step?

PATTERN	USE WHEN
Prompt → model	One answer is enough and a person checks it
Workflow (`WF`)	Steps can be drawn in advance
Agent (`AG`)	Next step depends on what it finds

- State the pattern and why. Patterns can mix (a workflow with an agent step).
- Name the model per AI step (`MOD` ID): a GenAI model, or a task-specific model when the task is narrow (e.g. classify a ticket, then let GenAI draft the reply).

6 Guardrails (G)

What must the system never let in, do, or say?

INPUT	EXECUTION	OUTPUT
Personal-data redaction, prompt-injection detection, topic scope	Argument checks, tool access by role, approval thresholds	Schema check, groundedness, leak filter, safe fallback

Each guardrail can raise false alarms: test it too.

7 Evals (E)

How do we prove it works, and still works after a change?

DIMENSION	METRIC	EVALUATOR	THRESHOLD
Correctness (accuracy, precision / recall for classifiers)		Code / LLM as judge / human	
Groundedness			
Format / policy			

- Test set: normal, hard, edge and adversarial cases.
- Code first, judge second, people to calibrate the judge.

8 Observability (O)

In production, can we see what it did, and why?

SIGNAL	WHAT IS LOGGED	ALERT / REVIEW TRIGGER
Traces (inputs, sources, tools, prompt & model version)		
Speed, errors, cost		
Quality (judge scores, user feedback)		

Every production failure becomes a test case.

9 Experiments (X)

Which design choice must be tested before we trust it?

HYPOTHESIS	VARIANT VS. BASELINE	DATASET	DECISION METRIC

Change one thing at a time on the same dataset; offline first, then a small live pilot.

10 Risks and open questions

What can go wrong here, and would anyone notice?

List failure modes, their impact, and assumptions still to validate.

04 TEMPLATE One-pager

The high-level solution a sponsor reads in two minutes. It summarises the project deck and adds nothing it does not support.

Header: `<system name>` · Owner · Version · Date

TOPICS 1 Problem 2 Solution 3 Risks and guards 4 Evals

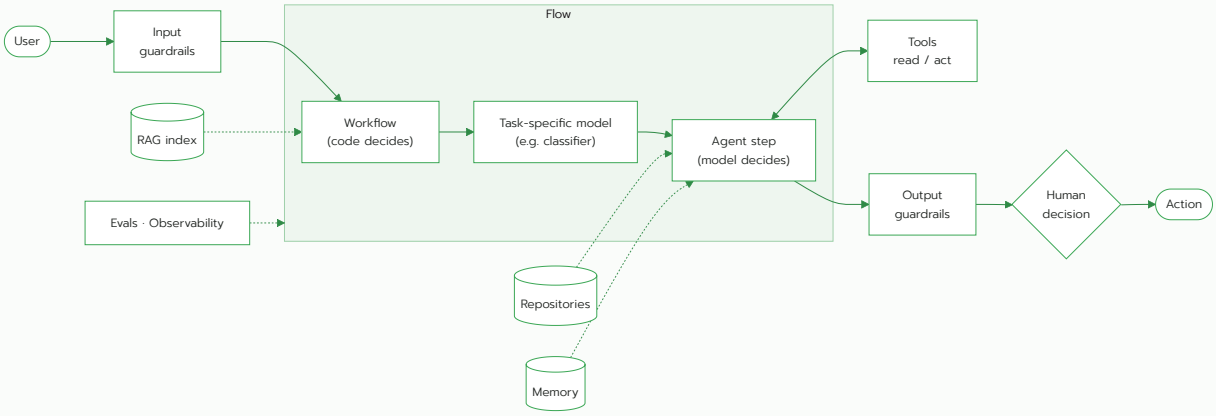
1 Problem

> **[User]** struggles to **[task]** because **[cause]**, resulting in **[measurable consequence]**.

Baseline · Scope · Non-goals: same as the project deck.

2 Solution

One hypothesis, one picture. Keep only the components your design uses.



COMPONENT	USED FOR	CATALOG ID
RAG index		RAG-01
Tool		TL-01

COMPONENT	USED FOR	CATALOG ID
Model (GenAI / task-specific)		MOD-01
Workflow / agent		WF-01, AG-01

#	USE CASE	TARGET USER	PRIORITY
UC1			★ riskiest

3 Risks and guards

RISK	GUARD

Three at most. A guard is a mechanism (code check, approval, threshold), not an intention. Add the **key assumption** in one line.

4 Evals

MEASURE	METRIC	BASELINE → TARGET / THRESHOLD
Value (success metric: time, cost, €)		
Quality (eval)		
Adoption		

End with **evidence so far** (proven vs. hypothesis) and the **decision asked**: what, from whom, by when.

CHECKLIST

05 Checklist

What to verify before you hand in, or when you challenge someone else's design. GenAI fails silently: outputs can look right and still be wrong.

Problem and value

- [ ] A real business problem: one process, one user group, one measurable business outcome.
- [ ] The problem names a user, a task, a cause and a measurable consequence.
- [ ] Baseline, target, scope and non-goals are stated.
- [ ] The AI approach fits each task (rules, task-specific model, GenAI, agent) and the choice is explained.
- [ ] Value outweighs running cost; regulatory constraints (GDPR, EU AI Act) are named.

Project deck

- [ ] The hypothesis has capability, users, task, baseline → target, risks and guards.
- [ ] Every success metric has a baseline, a target and a measurement method.
- [ ] One big-picture diagram shows the whole system.
- [ ] 3–5 use cases, prioritised, with the riskiest marked ★.
- [ ] Each risk has a guard; the key assumption is named, with an owner.
- [ ] Conclusions separate what is proven from what is still a hypothesis.

Each use case

- [ ] The story names a user, a trigger, an outcome and the value.
- [ ] The flow shows where a human decides and what happens when a step fails.
- [ ] Autonomy per step is stated; accountability sits with a person.
- [ ] Only the blocks it needs are used; skipped ones have a reason.
- [ ] Context sources have freshness and access rules; the output has a schema or is free text for a human.
- [ ] Tool arguments are validated; act tools are limited or approved.

- [ ] The flow is the simplest pattern that works, and each AI step names its model.
- [ ] Guardrails match the risk (input, execution, output).
- [ ] A test set (normal, hard, edge, adversarial) and a threshold per dimension.
- [ ] Any answer can be traced to its input, sources, tools, prompt and model version.
- [ ] Key design choices have an experiment with a decision metric.

Component catalog

- [ ] Every component is listed once, with ID, name, type and description.
- [ ] Every component is used by at least one use case; use cases refer to IDs.

One-pager

- [ ] Fits on one page.
- [ ] The diagram shows only the components used, and where a human decides.
- [ ] At most three risks, each with a concrete guard.
- [ ] Ends with a specific decision: what, who, by when.

Pitch

- [ ] Five slides; each title states a conclusion.
- [ ] Shows numbers: baseline, target, thresholds.

Prototype

- [ ] Addresses the riskiest use case.
- [ ] Proves one or two concepts (e.g. grounded answer, schema check, blocked attack, eval with threshold).

Consistency

- [ ] Project deck, pitch, one-pager and prototype tell the same story.
- [ ] Numbers, names and IDs match across all deliverables.

Peer challenge questions

01 Why AI here, why this approach (GenAI or task-specific model), and which simpler alternative was rejected?

02 Which assumption, if wrong, breaks the whole design?

03 Is the human review meaningful, or merely ceremonial?

04 What happens when the system is wrong, and would anyone notice?

05 What changes when use grows from 50 cases to 50,000, and does the cost still pay off?